Replication-lag

Wednesday, August 26, 2026 10:44 AM

. What actually causes replication lag? (The 4 Culprits)

Lag is the time difference between when a write commits on the leader and when it becomes visible on a follower. It is **never** just about network speed. Here are the real causes:

- **The Network RTT (The Red Herring):** This is the least common cause. Modern datacenter networks have sub-millisecond latency. Inter-region (e.g., US to Europe) adds ~100ms, which is trivial.
- **The I/O Bottleneck (The Heavy Lifter):** The leader writes to its disk sequentially (fast). The replica, however, must apply the change to its *existing* B-Tree or LSM-Tree indexes. This involves random disk I/O and updating secondary indexes. If the replica's disk IOPS are slower than the leader's, lag spikes.
- **The "Thundering Herd" of Reads (The Hidden Killer):** Replicas are usually used for analytical queries (heavy SELECT scans). When a replica is busy crunching a 5-minute analytical report, its CPU is maxed out. It stops applying the replication stream. The lag grows while it processes read queries.
- **Single-Threaded Apply (The Architectural Choke):** Most databases (MySQL, PostgreSQL) apply the replication stream **on a single CPU core** on the replica. This is to guarantee that transactions are applied in the exact same order as the leader. If the leader handles 10,000 writes/sec across 20 CPU cores, the replica is trying to replay those 10,000 writes/sec on *1 core*. It physically cannot keep up.

2. How is replication lag actually measured?

Databases do not use a "stopwatch." They measure lag using **transaction timestamps** embedded in the replication log.

- **MySQL:** Measures Seconds_Behind_Source. The replica compares the Unix timestamp of the *last event* it just applied, against the current timestamp on the replica's own system clock.
- **PostgreSQL:** Uses pg_stat_replication and calculates the write_lag, flush_lag, and replay_lag by comparing the leader's current LSN (Log Sequence Number) against the LSN the replica reports back.

Crucial operational trap: Seconds_Behind_Source is a **lie** if the replica is completely stalled (e.g., network partition). If the replica hasn't received *any* events in 5 minutes, it reports 0 lag because the last timestamp it applied was 5 minutes ago, matching its current system clock. The lag is actually infinite, but it reports zero. You must monitor **heartbeat timestamps** (empty writes) to catch this.

3. Why is lag the #1 operational headache?

Because it breaks the **mental model** developers have about databases. There are 3 catastrophic failure modes caused solely by lag:

- **Read-Your-Writes:** A user updates their profile picture. They hit "Save." The leader writes it. They refresh the page, and their read goes to a lagging replica that still shows the old picture. The user thinks the system is broken.
- **Monotonic Reads:** A user reads a comment thread on Replica A (up-to-date). They refresh and get routed to Replica B (lagging). The new comment they just saw *disappears*. Data goes backward in time for the user.
- **The "Zombie" Backup:** If the leader crashes and failover promotes a lagging replica to be the new leader, you have two choices: **(a)** Lose all the data that hadn't replicated yet, or **(b)** Bring the old leader back online as a replica, which then attempts to revert the new leader's data, causing split-brain and corrupting primary keys.

4. Addressing your statement: "To reduce this we can use the physical logic"

Let's dissect this because it's a common intuitive leap that is actually **incorrect**.

If by "physical" you mean **Physical (Row-Based) Replication** (shipping the exact changed rows), this *increases* bandwidth but *reduces* CPU overhead on the replica because the replica doesn't have to re-parse complex SQL logic.

However, if by "physical" you mean **Physical (Disk-Block) Replication** (shipping raw disk pages), this makes lag **worse**, not better. Here's why:
If you ship a raw disk page change to a replica, and the replica is running a heavy analytical query that has that exact same disk page locked in its cache, the replication apply process blocks. It waits. The lag spikes. **Physical replication is brittle and stalls easily.**

The Real Ways to Mitigate Lag (The "Ops" Toolkit)

Since you cannot eliminate lag, you manage it with these 4 proven mechanisms:

1. **Synchronous Replication (The Nuclear Option):** Make the client wait until the replica applies the change. This fixes lag by definition, but kills write throughput and availability. Only used for critical financial data.
2. **Read-Only Routing with "Bounded Staleness":** Configure your load balancer to *only* route read requests to replicas whose lag is under a threshold (e.g., < 1 second). If a replica lags beyond that, it is automatically removed from the read pool until it catches up.
3. **Session "Stickiness":** Route a single user's session to *exactly one* replica for the duration of their session. This prevents the "disappearing data" problem (Monotonic Reads), even if that replica lags.

4. **Scale the Replica's Hardware:** Since apply is single-threaded, give the replica faster **single-core CPU performance** and **NVMe SSDs** with higher IOPS than the leader. Leaders scale out (more cores); replicas scale *up* (faster cores).

The Ultimate Operational Truth About Lag

Lag is not a bug; it is a **thermometer** for the health of your entire stack.
When lag spikes, 90% of the time it is **not** a database problem—it is an **application problem**. It usually means your developers pushed a new schema migration (e.g., ALTER TABLE ADD COLUMN) that locks the table, or an analyst ran an unindexed SELECT * query that maxed out the replica's CPU.

Operationally, you treat lag like a pressure gauge: you don't try to stop the pressure (lag); you build alarms for when it exceeds 10 seconds, and you architect your application to gracefully tolerate temporary staleness using **Version Vectors** or **Hybrid Logical Clocks (HLCs)** in the application code itself.